Architecture.md v1.1

Chapter 17 — AI Model Management & Intelligence Orchestration

This chapter defines how AI OS interacts with Large Language Models (LLMs) and other AI providers. The objective is to ensure that AI OS remains **model-agnostic**, allowing models to be added, replaced, or upgraded without requiring architectural changes.

17.1 Purpose

The AI Model Management System shall:

- Support multiple AI providers.
- Route tasks to the most appropriate model.
- Optimize quality, speed, and cost.
- Provide automatic fallback mechanisms.
- Enable future local model execution.

The architecture shall never depend on a single AI provider.

17.2 Design Philosophy

AI models are **execution engines**, not architectural components.

Jarvis, Friday, and Worker Agents may request reasoning from an AI model, but the choice of model is determined by the Model Manager.

No component shall hard-code a dependency on a specific provider.

17.3 Model Manager

The Model Manager is a Core Service responsible for:

- Registering AI providers.
- Selecting models.
- Managing provider health.

- Monitoring usage.
- Executing fallback strategies.
- Recording model performance metrics.

The Model Manager does not perform reasoning itself.

17.4 Supported Providers

The architecture supports:

- OpenAI
- Google Gemini
- Anthropic Claude
- Local LLMs
- Future AI providers

Providers are interchangeable through configuration.

17.5 Model Selection Strategy

The Model Manager selects a model using:

- Task type
- Context length
- Required reasoning complexity
- Cost policy
- Latency requirements
- Provider availability

Example:

- Strategic reasoning → Large reasoning model.
- Simple automation → Fast, low-cost model.
- Image understanding → Vision-capable model.

17.6 Fallback Strategy

If the preferred provider is unavailable:

1. Detect failure.
2. Select compatible backup provider.

3. Retry the request.
4. Log the fallback event.

Users should experience minimal disruption.

17.7 Usage Policies

The Model Manager records:

- Token usage
- Cost estimates
- Response time
- Failure rate
- Success rate

These metrics support optimization and monitoring.

17.8 Local AI Support

Future versions may execute:

- Local language models.
- Offline reasoning.
- On-device embeddings.

Local execution shall use the same interfaces as cloud providers.

17.9 Context Management

Before sending a request to any model:

- Gather relevant memory.
- Remove unrelated context.
- Respect token limits.
- Protect sensitive information.

The Model Manager never sends unnecessary data.

17.10 Security

AI providers shall never receive:

- Raw API keys.
- Internal system secrets.
- Unapproved sensitive user information.

All outbound requests pass through the security and permission layers.

17.11 Future Expansion

Future versions may introduce:

- Automatic cost optimization.
- Dynamic multi-model collaboration.
- Specialized reasoning models.
- AI benchmarking.
- Fine-tuned domain models.

These additions must remain compatible with the Model Manager architecture.

End of Chapter 17

Architecture.md v1.1 Main Specification Complete